Uông Minh Khải – 1721031027

STT: 34

==========

I. System Architecture Diagram

Kiến trúc được chọn: 3-Tier (Layered Architecture) kết hợp MVC

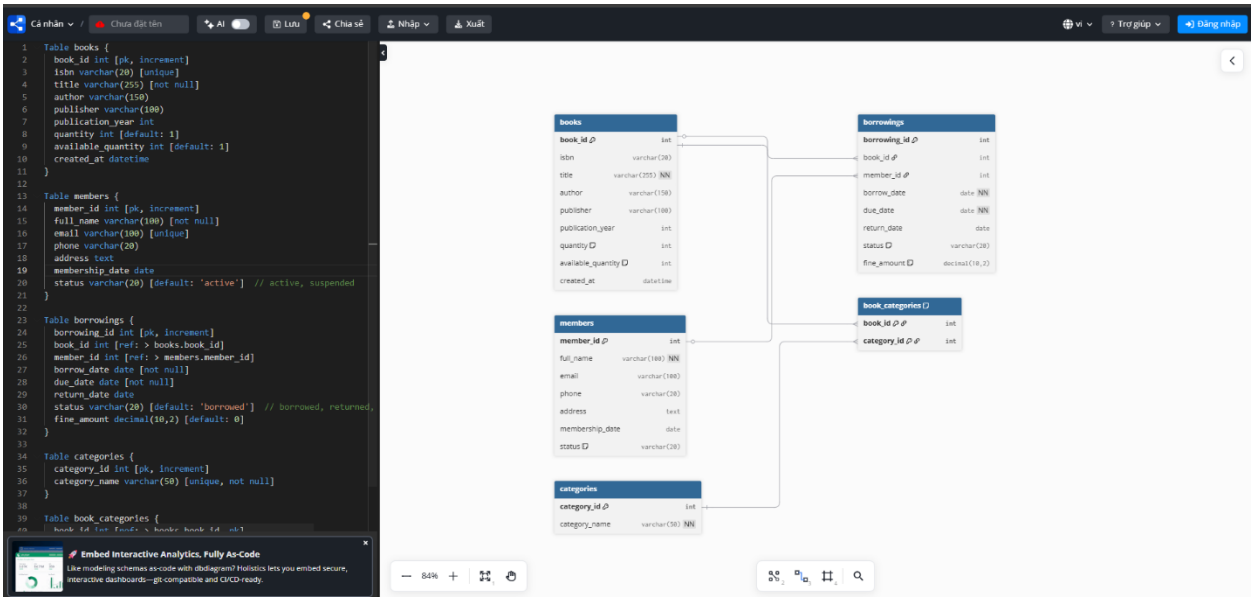
Mô tả sơ đồ:

- **Presentation Layer (Frontend)**: Giao diện người dùng (Web/App) cho Thủ thư, Quản lý thư viện và Độc giả. (Ví dụ: Trang quản lý sách, mượn/trả sách, tìm kiếm sách, báo cáo)
- **Application Layer (Backend)**:
  - Controller: Nhận request từ người dùng
  - Service Layer: Xử lý business logic (kiểm tra sách còn hay không, tính phí phạt, kiểm tra hạn mượn...)
  - Repository / DAO Layer: Truy vấn database
- **Data Layer**: Database (MySQL / PostgreSQL)
- **Các thành phần bổ sung**: Authentication & Authorization (quyền thủ thư / quản lý / độc giả), Logging, Notification (nếu có gửi email nhắc trả sách).

Lý do chọn kiến trúc này:

- Dễ phân tầng, tách biệt rõ ràng giữa giao diện, logic nghiệp vụ và dữ liệu → dễ bảo trì và mở rộng.
- Phù hợp với mini project nhóm: thành viên có thể phân công rõ (một người làm frontend, một người làm backend, một người làm database).
- Dễ áp dụng các Design Pattern (MVC, Repository, Service).
- Hỗ trợ nhiều người dùng đồng thời (thủ thư + độc giả).

II. ERD trên dbdiagram.io



Code:

Table books {

book_id int [pk, increment]

isbn varchar(20) [unique]

title varchar(255) [not null]

author varchar(150)

publisher varchar(100)

publication_year int

quantity int [default: 1]

available_quantity int [default: 1]

created_at datetime [default: current_timestamp]

}

Table members {

member_id int [pk, increment]

```
  full_name varchar(100) [not null]
  email varchar(100) [unique]
  phone varchar(20)
  address text
  membership_date date [default: current_date]
  status varchar(20) [default: 'active']  // active, suspended
}

Table borrowings {
  borrowing_id int [pk, increment]
  book_id int [ref: > books.book_id]
  member_id int [ref: > members.member_id]
  borrow_date date [not null]
  due_date date [not null]
  return_date date
  status varchar(20) [default: 'borrowed']  // borrowed, returned, overdue
  fine_amount decimal(10,2) [default: 0]
}

Table categories {
  category_id int [pk, increment]
  category_name varchar(50) [unique, not null]
}
```

Table book_categories {

book_id int [ref: > books.book_id, pk]

category_id int [ref: > categories.category_id, pk]

}

III. SQL Schema — CREATE TABLE + Seed Data

Tạo 2 file:

CodeERD.sql	3/27/2026 11:39 AM	Microsoft SQL Ser...	2 KB
schema.sql	3/27/2026 11:40 AM	Microsoft SQL Ser...	2 KB
seed_data.sql	3/27/2026 11:41 AM	Microsoft SQL Ser...	2 KB
34_1721031027_UongMinhKhai_TX2.docx	3/27/2026 11:42 AM	Microsoft Word D...	172 KB

IV. Design Patterns sẽ áp dụng

Danh sách Design Patterns (tạo file design_patterns.md):

1. **MVC Pattern (Model-View-Controller)**
    - Model: Đại diện cho dữ liệu (Books, Members, Borrowings).
    - View: Giao diện người dùng.
    - Controller: Xử lý request và gọi Service. → Giúp tách biệt rõ ràng, dễ phát triển và bảo trì.
2. **Repository Pattern**
    - Tách biệt logic truy vấn database ra khỏi business logic. Mỗi entity có một Repository (BookRepository, MemberRepository...). → Dễ thay đổi database sau này và dễ viết unit test.
3. **Service Layer Pattern**
    - Đặt toàn bộ business logic vào Service (ví dụ: BookService, BorrowingService). → Kiểm tra sách còn không, tính phí phạt, kiểm tra hạn mượn...
4. **Factory Pattern** (tùy chọn)
    - Dùng để tạo các loại báo cáo (báo cáo sách mượn nhiều, sách quá hạn...).
5. **Singleton Pattern**

- Áp dụng cho Database Connection hoặc Configuration Manager.

Lý do áp dụng: Các pattern này giúp code sạch, dễ mở rộng, dễ phân công nhóm và phù hợp với ứng dụng quản lý dữ liệu như thư viện.